

ARCHITECTURALE STIJLEN

		Partitionering	
		Technisch	Op basis domein
Deployment	Monolitisch	• gelaagd • microkernel	modulaire monoliet
	Gedistribueerd	event-driven	microservices

- technische partitionering
 - presentatie
 - logica
 - persistentie
- partitionering op basis domein
 - klanten
 - betalingen
 - verzendingen

"Mag ik bij opdeling volgens domein geen opdeling in presentatie/logica/persistentie toepassen?"

STIJL: GELAAGDE ARCHITECTUUR

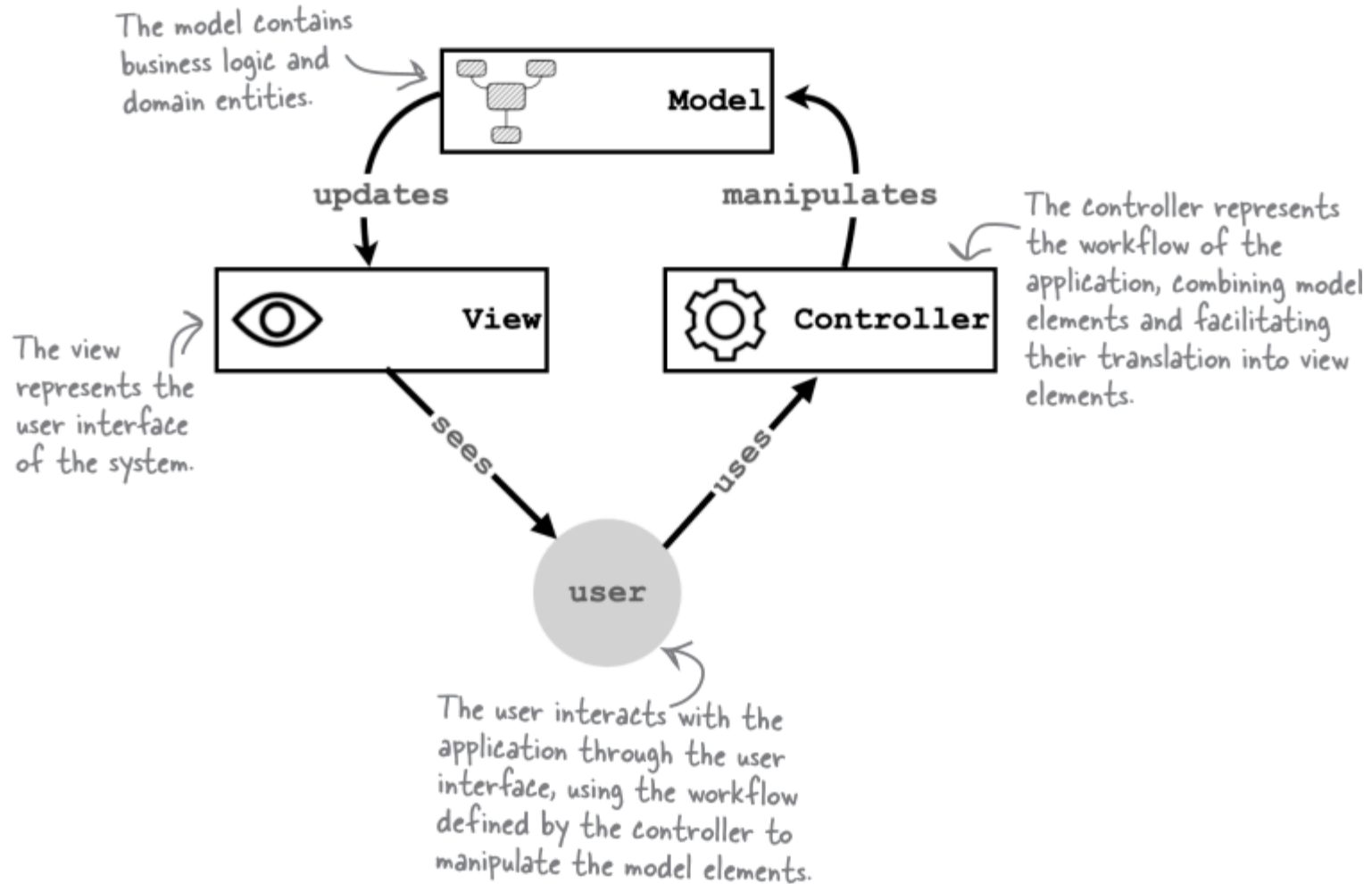
- technische partitionering
- monolithisch

CASE: NAAN & POP

REQUIREMENTS

- online bestellingen kunnen plaatsen
- snel online
- scheiding van technische onderdelen
- simpel (maar nog aanpasbaar)

VERBAND MET MVC



LAGEN IN DJANGO

```
from django.db import models

class Project(models.Model):
    title = models.CharField(max_length=100)
    description = models.TextField()
    technology = models.CharField(max_length=20)
```

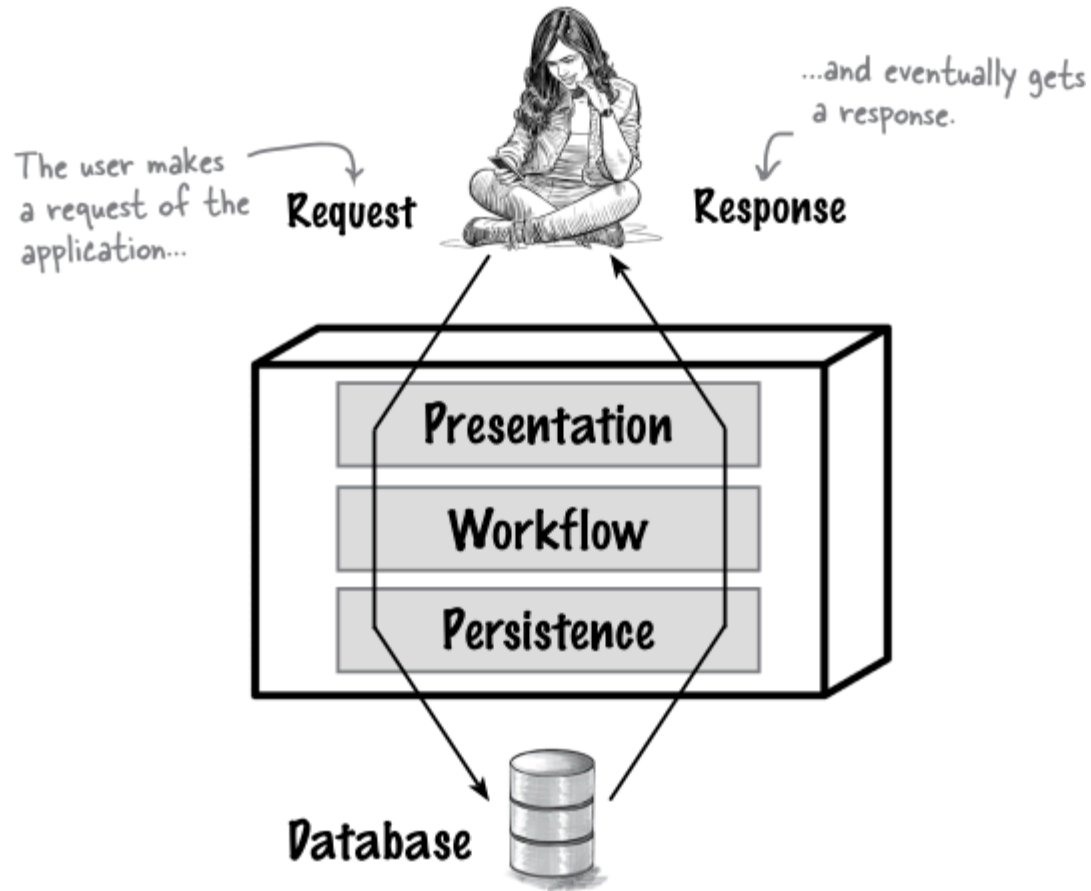
```
{% extends "base.html" %}

{% block page_content %}
<h1>Projects</h1>
<div class="row">
{% for project in projects %}
    <div class="col-md-4">
        <div class="card mb-2">
            <div class="card-body">
                <h5 class="card-title">{{ project.title }}</h5>
                <p class="card-text">{{ project.description }}</p>
                <a href="{% url 'project_detail' project.pk %}"
                    class="btn btn-primary">
                    Read More
                </a>
            </div>
        </div>
    </div>
{% endfor %}
</div>
</div>
</div>
```

```
from django.shortcuts import render
from projects.models import Project

def project_index(request):
    projects = Project.objects.all()
    context = {
        "projects": projects
    }
    return render(request, "projects/project_index.html", context)
```

INTERACTIE MET GEBRUIKER (1)



INTERACTIE MET DE GEBRUIKER

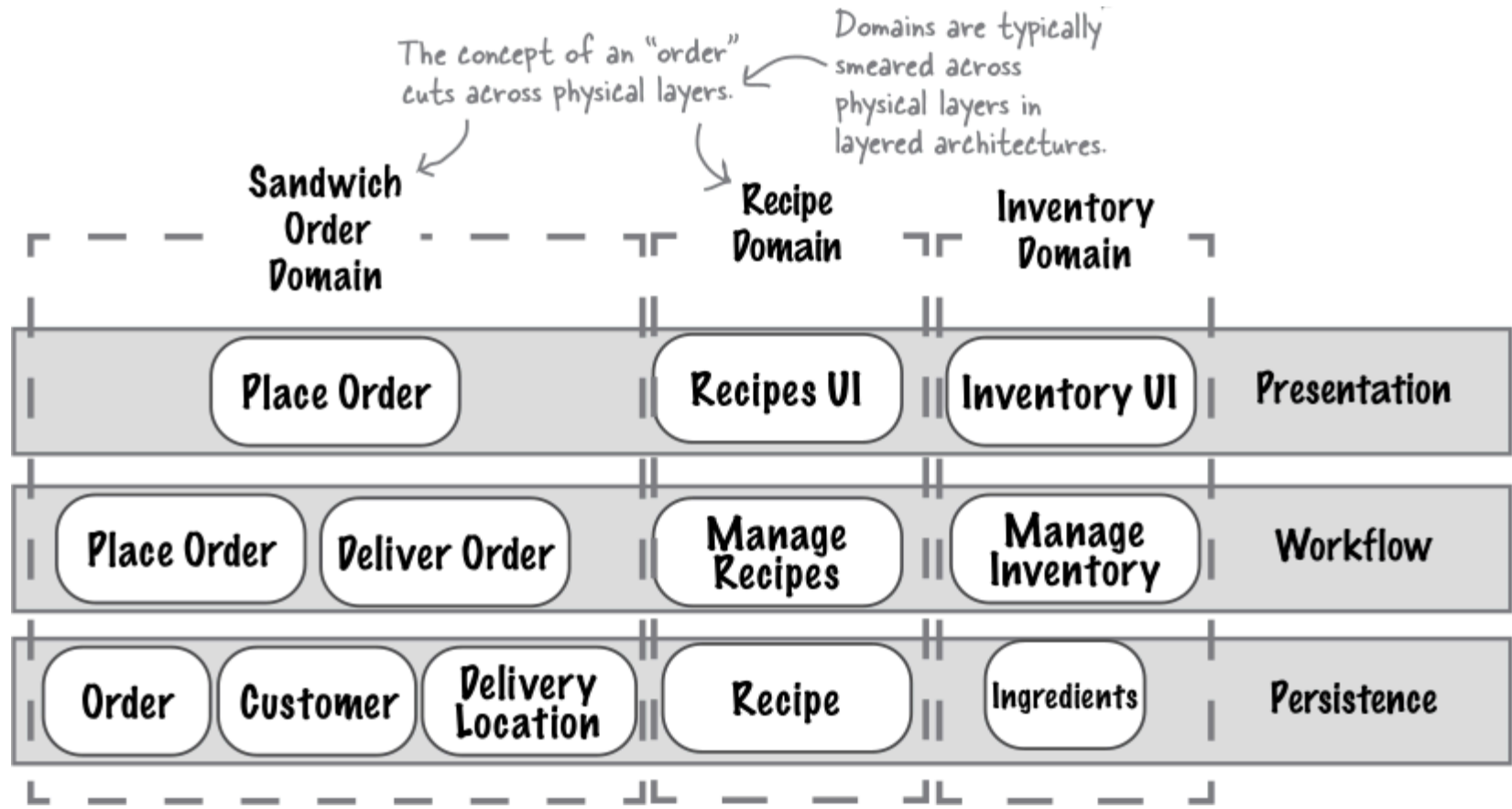
(2)

```
def UI_layer(request):  
    data = request.get_data()  
    return business_logic_layer_function(data) # terug naar gebruiker  
  
def business_logic_layer_function(data):  
    processed_data = process_data(data) # binnen deze laag  
    return data_access_layer_function(processed_data) # terug naar U  
  
def data_access_layer_function(data):  
    retrieve_data = retrieve_data(data) # bijvoorbeeld uit de databa  
    return retrieved_data # terug naar business logic layer
```

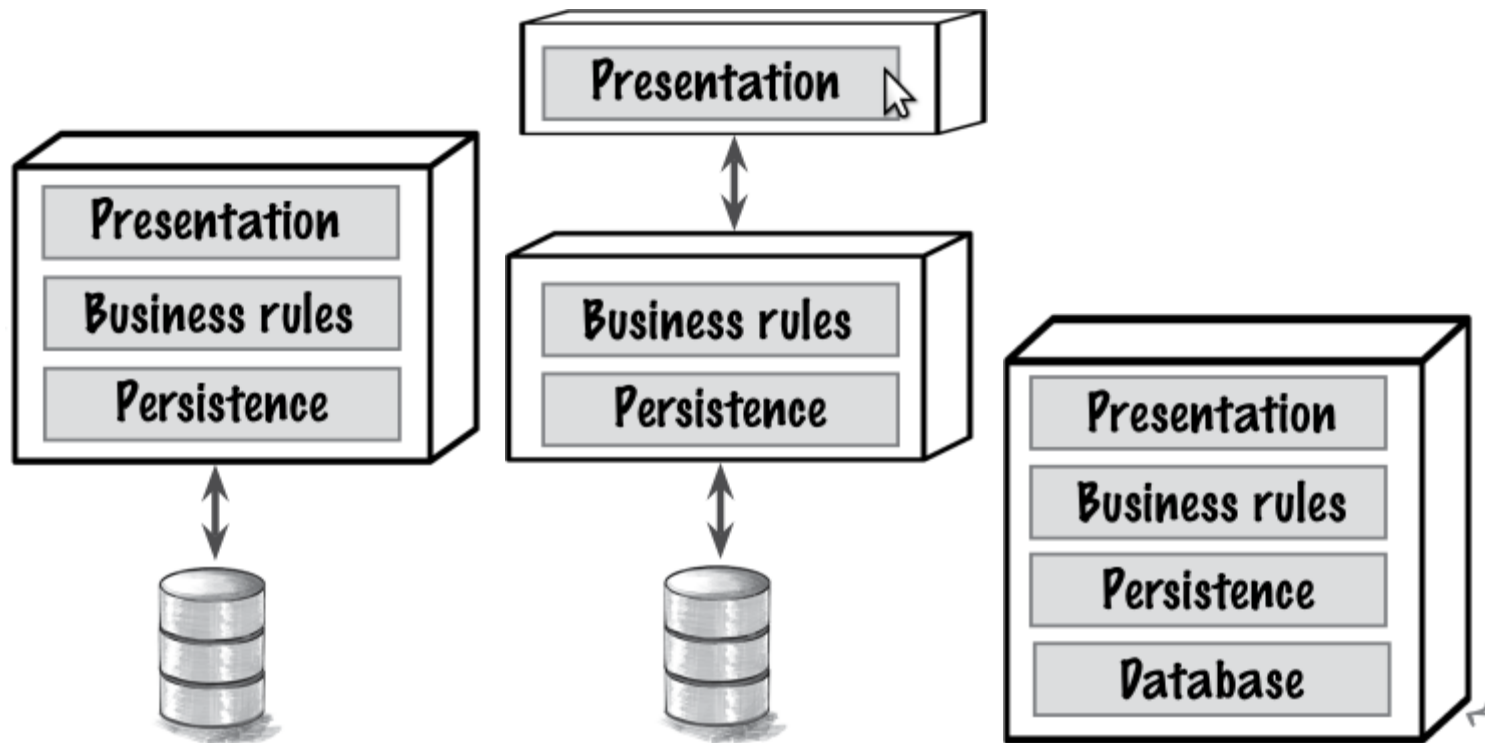

OPDELING LOGISCHE COMPONENTEN

- code wordt typisch georganiseerd per laag
- groepering per logische component hier secundair

VOORBEELD OPDELING



VERSCIJNINGSVORMEN (FYSIEKE ARCHITECTUREN)



TECHNISCHE WIJZIGINGEN VS. DOMEINWIJZIGINGEN

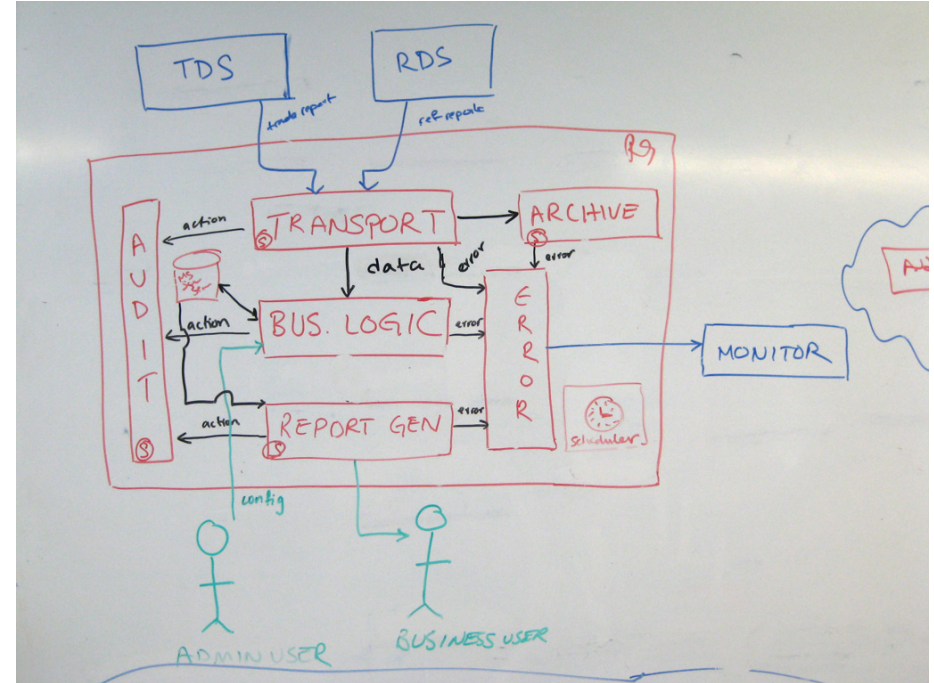
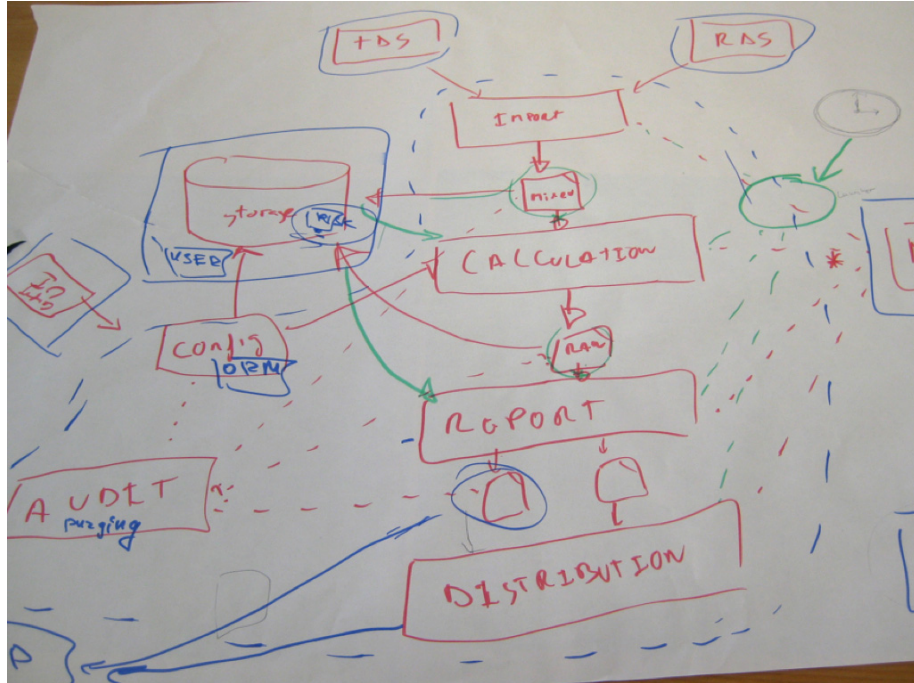
- zijn vertrokken uit het domein
- lagen hebben domein technisch opgedeeld
- technische aanpassingen zijn relatief makkelijk
- domeinwijzigingen zijn ingrijpend

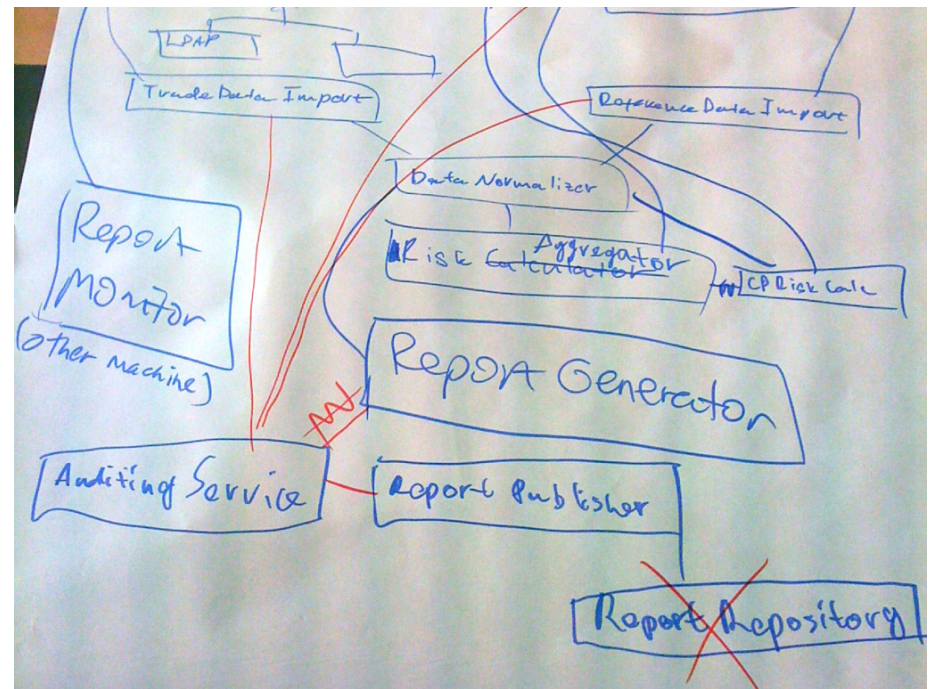
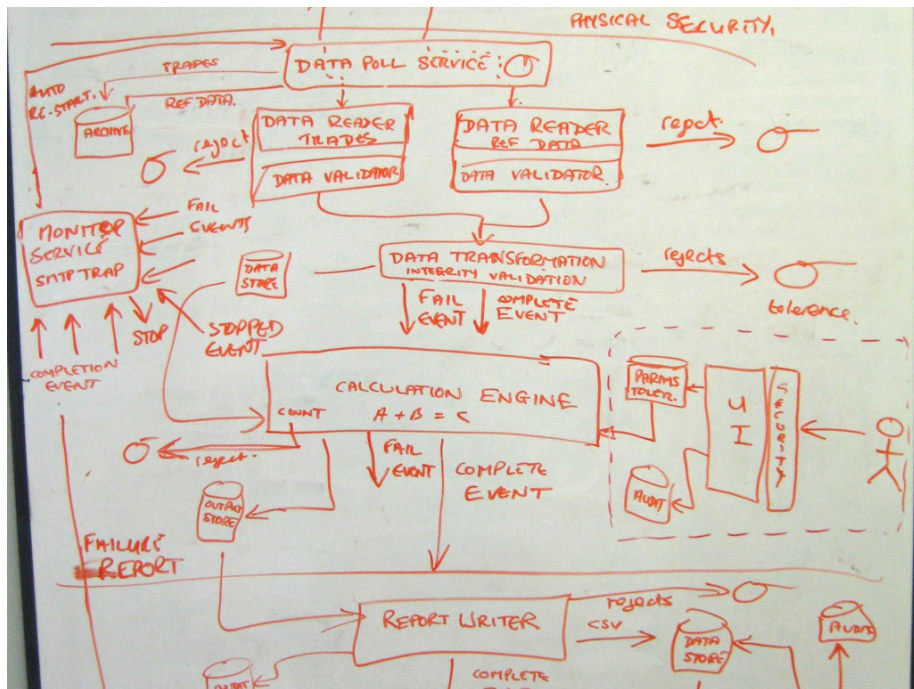
STERKE EN ZWAKKE PUNTEN

-  praktisch haalbaar
-  performant
-  handig voor technologiespecialisten
-  goedkoop
-  beperkt schaalbaar
-  lastig aanpassingen te deployen
-  spreiding domein
-  kwetsbaar
-  vrij lastig te testen*

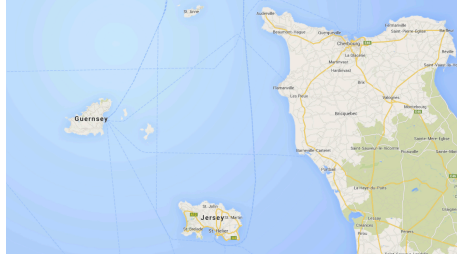
C4-MODEL

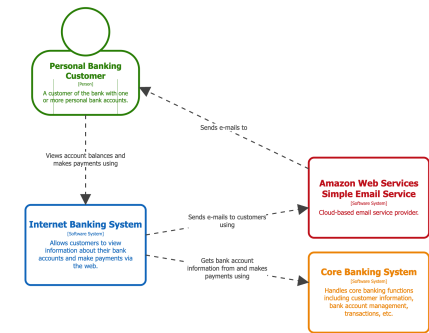
TYPISCHE DIAGRAMMEN



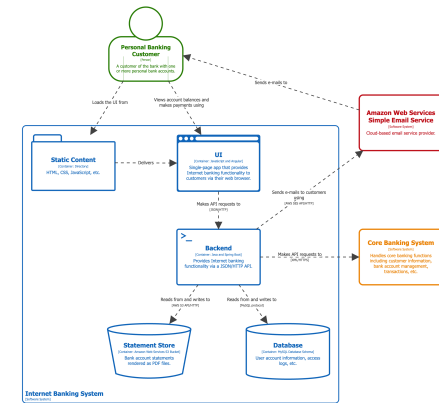


HIËRARCHISCHE VIEW

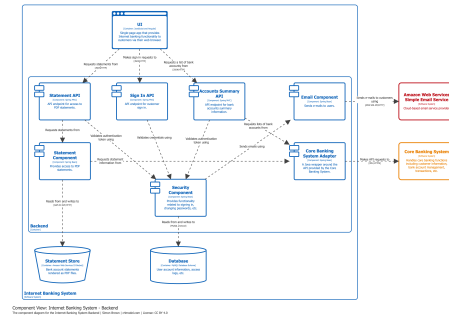




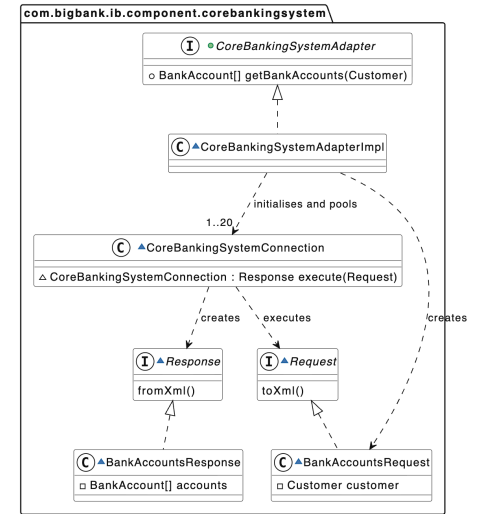
System Context View: Internet Banking System
The system context diagram for a fictional Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0



Container View: Internet Banking System
The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0



Component View: Internet Banking System - Backend - Core Banking System
A summary of the implementation details for the Core Banking System Adapter component | Simon Brown |



Code View: Internet Banking System - Backend - Core Banking System
A summary of the implementation details for the Core Banking System Adapter component | Simon Brown |

demo

SYSTEMCONTEXT

CONTAINERDIAGRAM

COMPONENT DIAGRAM

CODE DIAGRAM

DEPLOYMENT DIAGRAM

HULPMIDDELEN

- [abstractions](#)
- [review checklist](#)
- [FAQ](#)
- [tools](#)

TE KENNEN/GEBRUIKEN

- betekenis van de 4 abstracties
- notatie voor de eerste 3 abstracties
- deployment diagram
- uitleg queues en topics
- (later) toepassing op microservices